

Система контроля версий Git

Версионирование

Чтобы лучше понять проблему версионирования, рассмотрим пример дизайнера, который закончил работать над проектом и отправил финальную версию заказчику. У дизайнера есть папка, в которой хранится финальная версия проекта:

```
source/ barbershop_index_final.psd
```

Всё хорошо, дизайнер закончил работу, но заказчик прислал в ответ правки. Чтобы была возможность вернуться к старой версии проекта, дизайнер создал новый файл `barbershop_index_final_2.psd`, внёс изменения и отправил заказчику:

```
source/ barbershop_index_final.psd barbershop_index_final_2.psd
```

Этим всё не ограничилось, в итоге структура проекта разрослась и стала выглядеть так:

```
source/ barbershop_index_final.psd barbershop_index_final_2.psd ... barbershop_index_final_19.psd ... barbershop_index_latest_final.psd barbershop_index_latest_final_Final.psd
```

В разработке создавать новые файлы для версионирования - плохая практика. Обычно у разработчиков в папке проекта хранится множество файлов. Также над одним проектом может работать несколько человек. Если каждый разработчик для версионирования будет создавать новый файл, немного изменяя название предыдущей версии, то в скором времени в проекте начнётся хаос и никто не будет понимать, какие файлы нужно открывать.

Git

Программа, которая позволяет хранить разные версии одного и того же документа, переключаться между ранними и поздними вариантами, вносить и отслеживать изменения.

Работа с Git ведётся через командную строку. В разных операционных системах свои программы для взаимодействия с Git.

В Windows их две: PowerShell и cmd.exe. В Ubuntu это Terminal. Самая популярная программа на macOS тоже называется Terminal.

Подобные программы еще называют «терминал» или «консоль». Мы вводим команду и получаем реакцию машины: сообщение об ошибке, запрос на подтверждение информации, результат выполненных действий.

Установка в Windows

Скачайте exe-файл с [сайта Git](#) и запустите его. Это Git для Windows, он называется msysGit. Установщик спросит добавлять ли в меню проводника возможность запуска файлов с помощью Git Bash (консольная версия) и GUI (графическая версия). Подтвердите действие, чтобы далее вести работу через консоль в Git Bash. Остальные пункты можно оставить по умолчанию.

Установка в Linux

Используйте обычный менеджер пакетов вашего дистрибутива. Откройте терминал и введите подходящие команды.

- Если у вас 21 или более ранняя версия Fedora, используйте `yum install git`.
- Для 22 и последующих версий Fedora вводите `dnf install git`.
- Для дистрибутивов, основанных на Debian, например, Ubuntu, используйте `apt-get: sudo apt-get install git`.

Полный список команд для различных дистрибутивов можно посмотреть [здесь](#).

Установка на macOS

1. Скачиваем Git со страницы [проекта](#).
2. Запускаем загруженный файл.
3. Система может показать окно с ошибкой, где будет написано, что файл скачан с неавторизованного сайта и инсталлятор не может быть запущен. В таком случае нужно зайти в «Системные настройки» — «Безопасность» (Security and Privacy), в появившемся окне будет сообщение об ошибке и кнопка Open anyway (Всё равно открыть). Нажимаем.

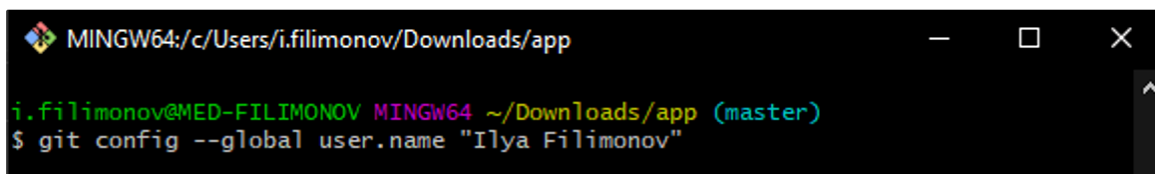
4. Система покажет окно, уточняющее хотите ли вы запустить установку.
Подтверждаем действие.
5. Установщик проведёт через все необходимые шаги.

Первоначальная настройка Git

Работа с любой программой всегда начинается с её настройки. Git можно настроить один раз и менять что-то только по мере необходимости.

Установка имени пользователя, от которого будут идти коммиты.

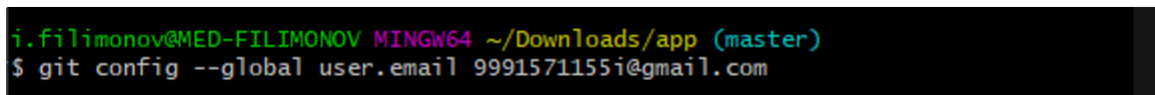
```
git config --global user.name "User Name"
```



```
MINGW64:/c/Users/i.filimonov/Downloads/app
i.filimonov@MED-FILIMONOV MINGW64 ~/Downloads/app (master)
$ git config --global user.name "Ilya Filimonov"
```

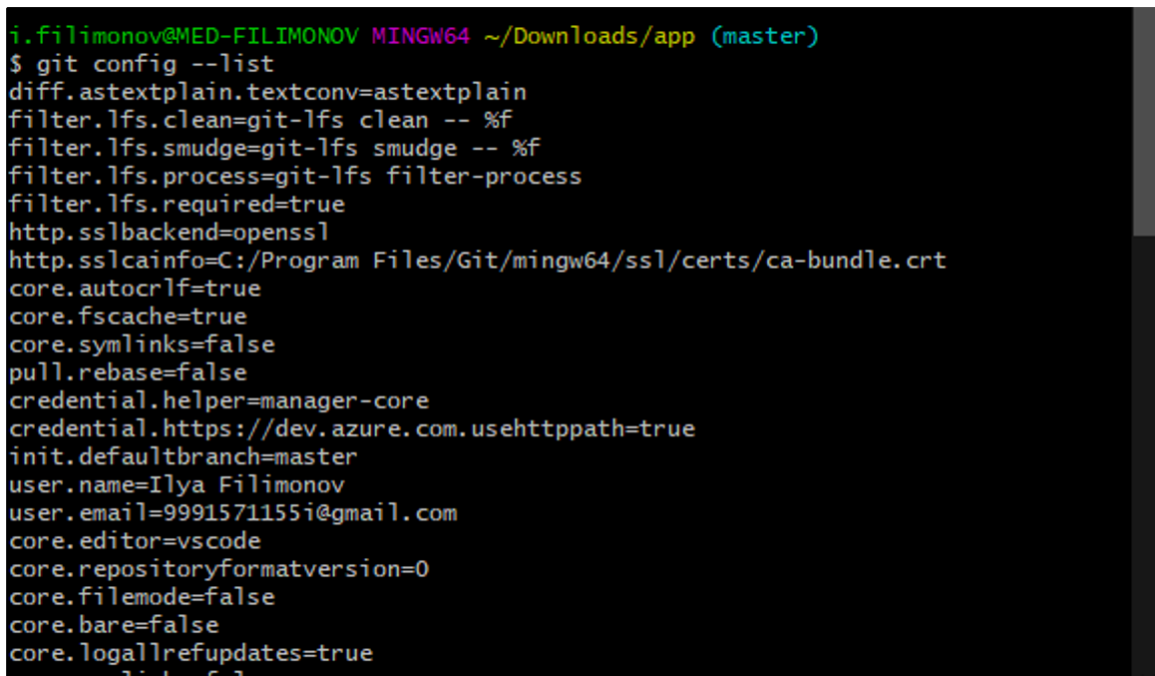
Установка адреса электронной почты. Обратите внимание, что адрес должен совпадать с тем, на который зарегистрирован аккаунт в Гитхабе.

```
git config --global user.email mail@gmail.com
```



```
i.filimonov@MED-FILIMONOV MINGW64 ~/Downloads/app (master)
$ git config --global user.email 9991571155i@gmail.com
```

С помощью команды `git config --list` можно посмотреть список всех установленных настроек.



```
i.filimonov@MED-FILIMONOV MINGW64 ~/Downloads/app (master)
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager-core
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Ilya Filimonov
user.email=9991571155i@gmail.com
core.editor=vscode
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
```

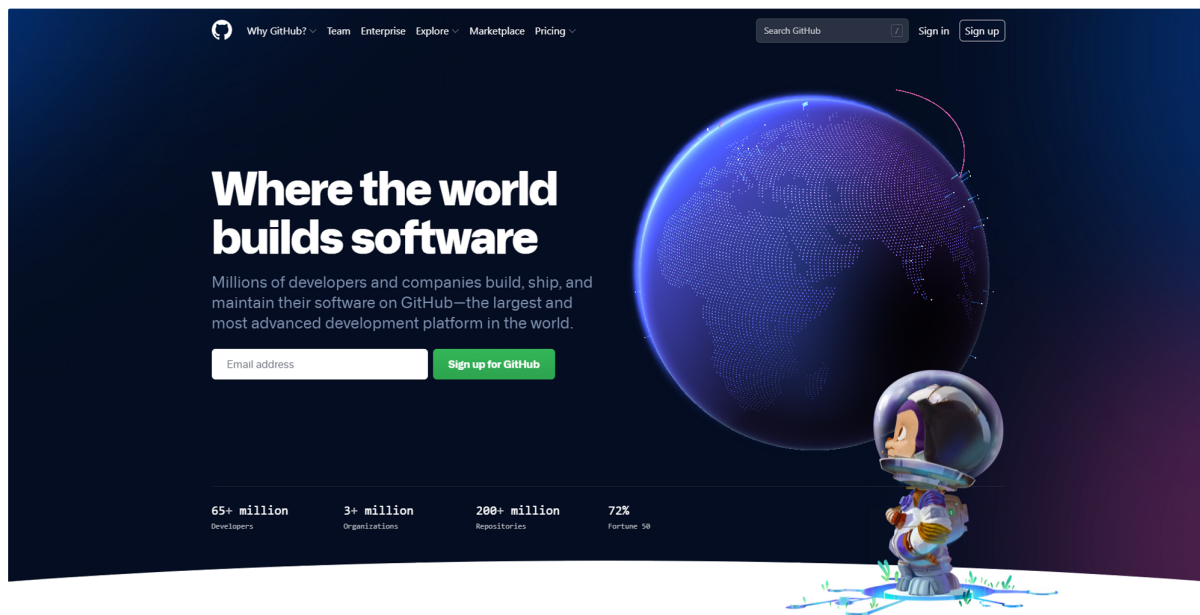
Регистрация на GitHub

Что такое GitHub?

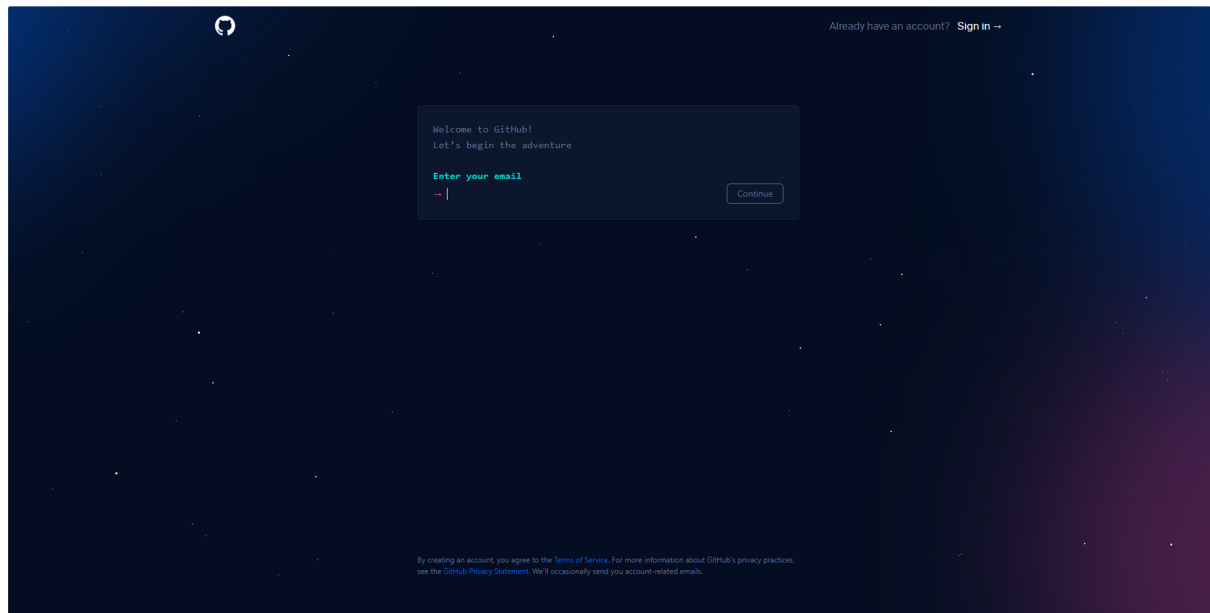
GitHub - веб-сервис, который основан на системе Git. Это социальная сеть для разработчиков, которая помогает удобно вести коллективную разработку IT-проектов. В ней можно публиковать и редактировать свой код и т.д.

Чтобы начать работу с GitHub, нужно зарегистрироваться на сайте.

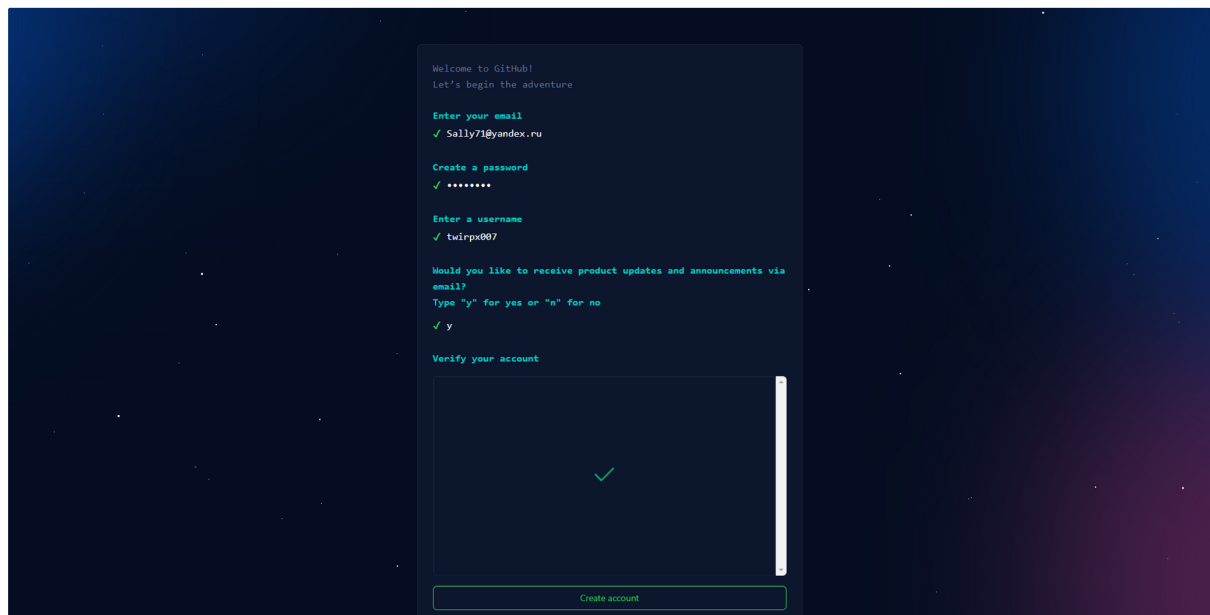
1. Переходим на [сайт GitHub](#).



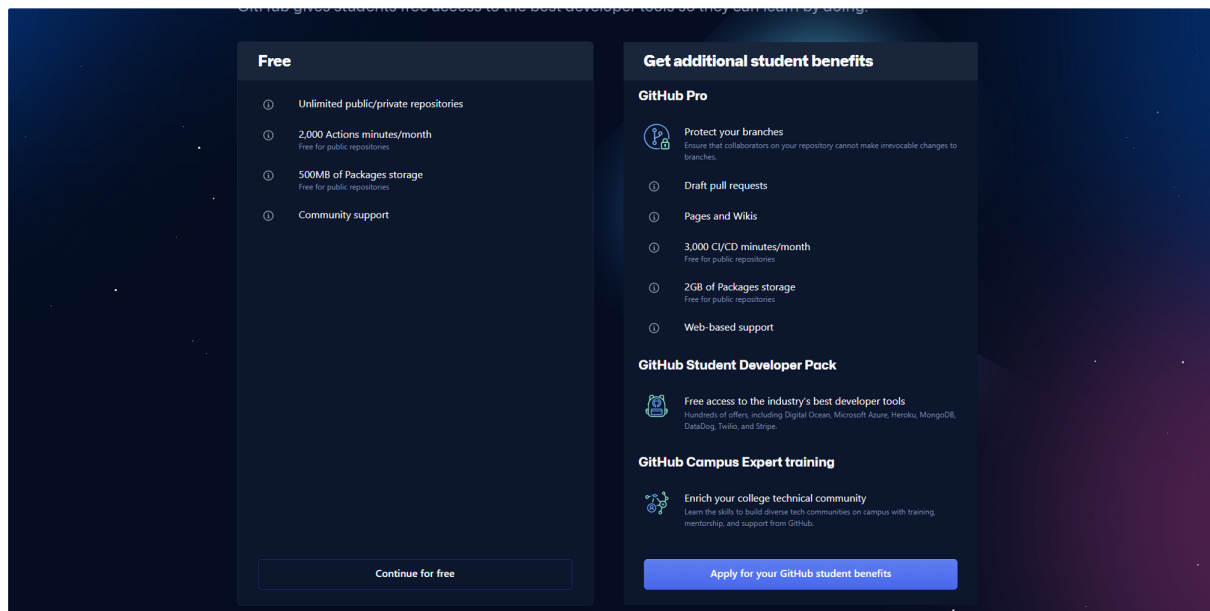
2. Нажимаем кнопку Sign up (зарегистрироваться), попадаем на страницу регистрации, где вводим обязательные данные: имя пользователя, адрес электронной почты и пароль. После заполнения полей проходим верификацию.



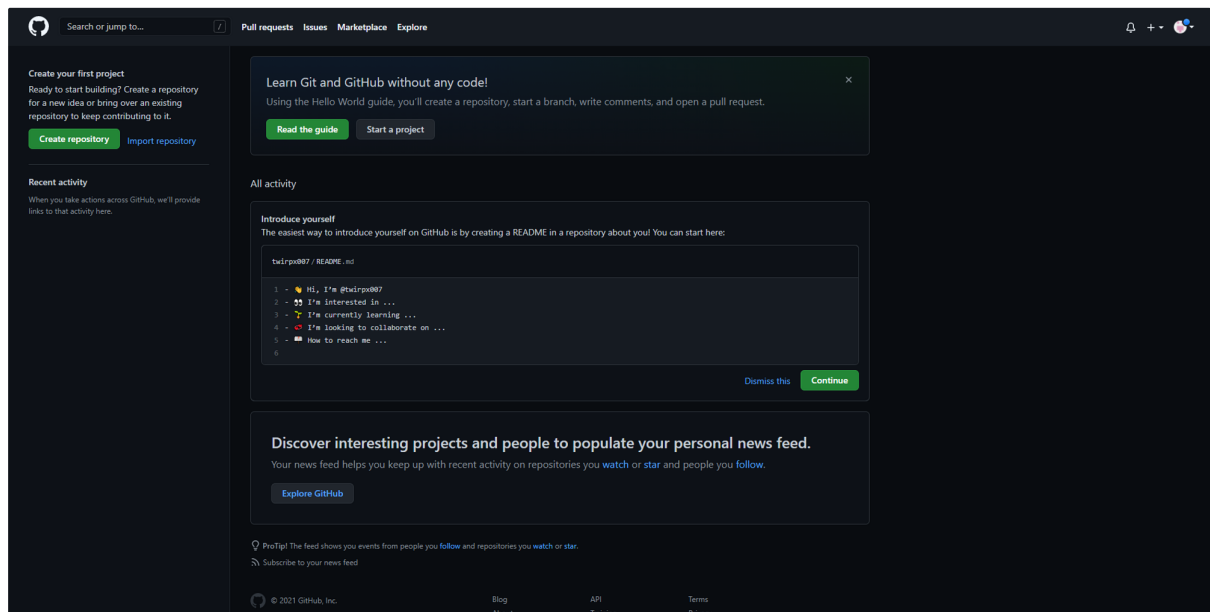
3. После заполнения данных и успешного прохождения верификации нажимаем на кнопку Create account.



4. После прохождения всех этапов на сайте, на указанный при регистрации ящик вам придёт письмо от GitHub. Откройте его и скопируйте код, который пришел на почту.
5. Далее отвечаем на вопросы от Github и кликаем на кнопку слева Continue for free.



6. После верификации GitHub предложит создать новый репозиторий, организацию или узнать больше о GitHub. Создаем новый репозиторий.



Теперь у вас есть профиль на Github и репозиторий для работы.

Устанавливаем SSH-ключи

Git установлен, профиль на GitHub создан. Осталось добавить SSH-ключ и можно приступать к работе с проектом.

Что такое **SSH-ключ** и зачем он нужен?

Чтобы работать со своего компьютера с GitHub, иметь доступ к проектам, хранящимся на сервисе, выполнять команды в консоли без постоянного подтверждения пароля, нужно пройти авторизацию у сервера. В этом помогают SSH-ключи.

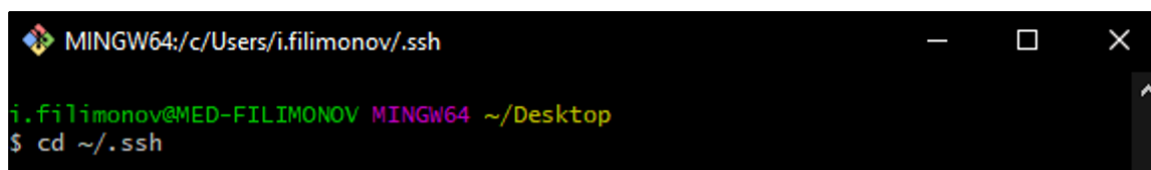
Каждый SSH-ключ содержит пару: открытый (публичный) и закрытый (приватный) ключ. Открытый ключ отправляется на сервер, его можно не прятать от всех и не переживать, что кто-то его увидит и украдёт. Он бесполезен без своей пары — закрытого ключа. А вот закрытый ключ — секретная часть. Доступ к нему должен быть только у вас.

Вы отправляете какую-то информацию на сервер, где хранится ваш публичный ключ, сервер понимает, что вы это вы, то есть идентифицирует именно вас, и даёт вам какой-то ответ. И только вы можете расшифровать этот ответ, потому что только у вас есть подходящий закрытый ключ. Получается что-то вроде связки логин-пароль только намного безопасней. Ваш пароль кто-то может узнать или подобрать, а чтобы получить ваш приватный SSH-ключ, злоумышленнику придётся взломать ваш компьютер.

Чтобы пройти авторизацию по SSH-ключу, его надо сгенерировать или найти уже ранее созданный ключ на своём компьютере.

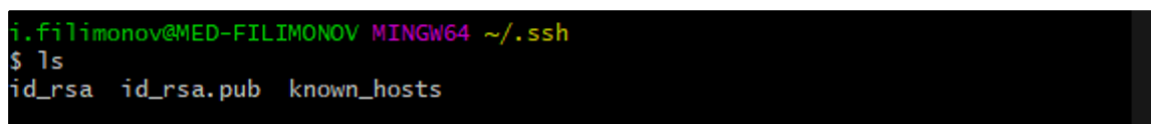
Сначала проверим, есть ли уже на компьютере ключ. По умолчанию SSH-ключи хранятся в каталоге `~/.ssh`, поэтому нужно проверить содержимое этого каталога.

- Открываем консоль.
- Вводим `cd ~/.ssh`, чтобы перейти в нужный каталог.



```
MINGW64:/c/Users/i.filimonov/.ssh
i.filimonov@MED-FILIMONOV MINGW64 ~/Desktop
$ cd ~/.ssh
```

- Используем `ls`, чтобы увидеть список всех файлов в каталоге.



```
i.filimonov@MED-FILIMONOV MINGW64 ~/.ssh
$ ls
id_rsa  id_rsa.pub  known_hosts
```

Ищем пару файлов с названиями вида имя и имя.pub. Файл с расширением .pub - ваш публичный ключ, а второй - ваш приватный, секретный ключ.

 Если таких файлов или даже каталога .ssh у вас нет, вы можете их сгенерировать.

Открываем консоль и вводим команду:

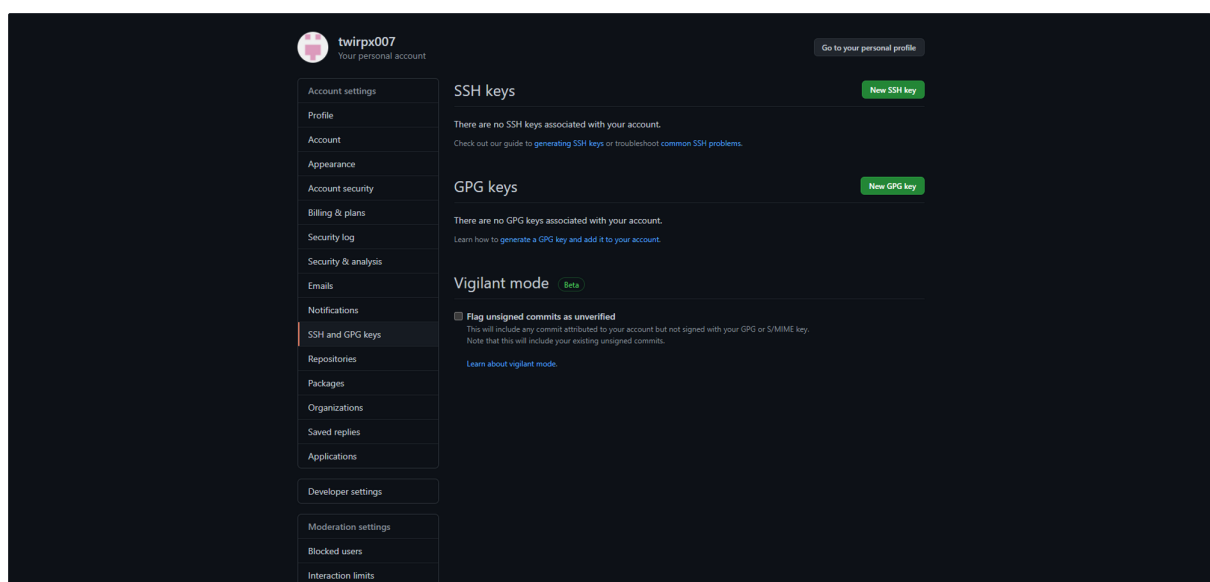
```
ssh-keygen -t rsa -b 4096 -C "your_mail@example.com"
```

Указываем тот адрес электронной почты, который вводили при регистрации на GitHub.

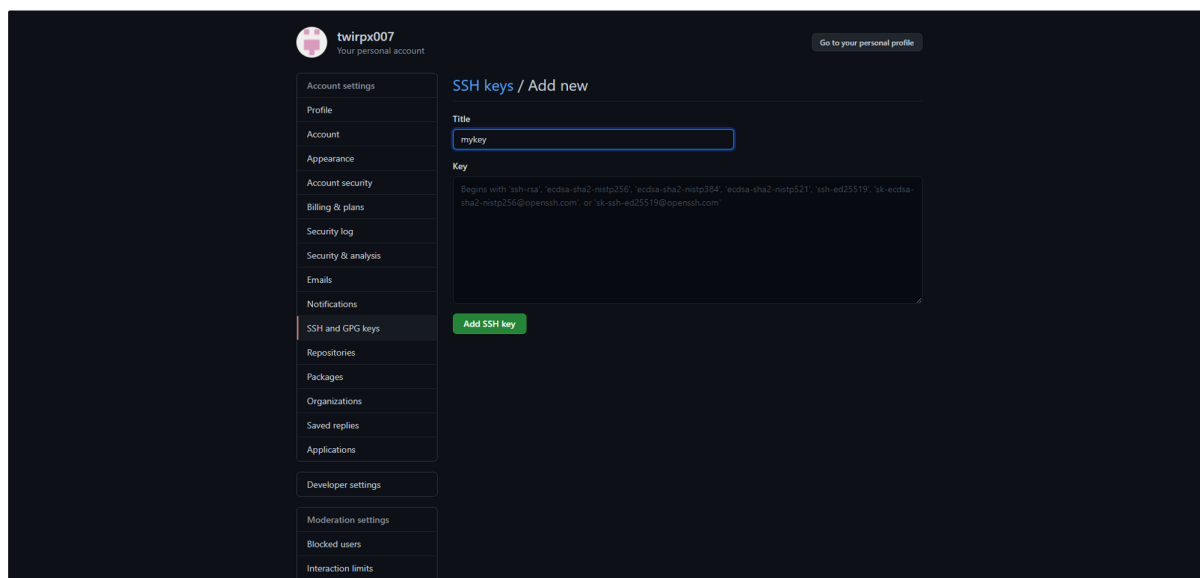
- Далее нужно указать расположение файла для сохранения ключа. Если вы не введёте путь до файла и просто нажмёте «Enter», ключ сохранится в файле, указанном в скобках.
- Теперь нужно установить пароль к вашему ключу и дважды ввести его. Если вы не хотите вводить пароль каждый раз, когда используете ключ, пропустите этот шаг, нажав «Enter», и ничего не вводите.
- После того как создан ключ, его нужно добавить на GitHub. Для этого копируем его содержимое с помощью одной из следующих команд:
 - Если вы на Windows `clip .`
 - Для пользователей macOS `pbcopy .`
 - На Linux используйте `sudo apt-get install xclip`, чтобы установить необходимый для копирования пакет `xclip`, а затем введите `xclip -sel clip .` Или введите команду `cat ~/.ssh/id_rsa.pub`, контент документа появится прямо в консоли и вы сможете скопировать ключ оттуда.

Можно пойти другим путём, открыть файл `id_rsa.pub` прямо в папке и просто скопировать содержимое оттуда.

- Переходим на страницу для работы с ключами в вашем профиле на GitHub.



Нажимаем кнопку New SSH key (новый SSH-ключ). Вводим имя ключа (можно придумать абсолютно любое) в поле Title (название), а в Key (ключ) вставляем сам ключ из буфера обмена. Теперь нажимаем Add SSH key (добавить SSH-ключ).



Если всё сделано верно, в списке появится новый ключ. Теперь можно начать работу с проектом.

Работа с репозиториями

Репозиторий - это рабочая директория с вашим проектом. Т.е. это та же папка с HTML, CSS, JavaScript и прочими файлами, что хранится у вас на компьютере, но находится на сервере GitHub. Поэтому вы можете работать с проектом удалённо на любой машине, не переживая, что какие-то из ваших файлов потеряются — все данные будут в репозитории при условии, что вы их туда отправите.

Если над проектом трудится команда разработчиков, как правило, создаётся общий репозиторий, в котором находится рабочая версия проекта (назовём его мастер-репозиторий). При этом каждый пользователь клонирует себе в профиль оригинальный репозиторий и работает именно с копией. Такая копия называется форком. Так как **форк** — ваша персональная версия мастер-репозитория, в нём вы можете пробовать разные решения, менять код и не бояться что-то сломать в основной версии проекта.

Как сделать форк мастер-репозитория?

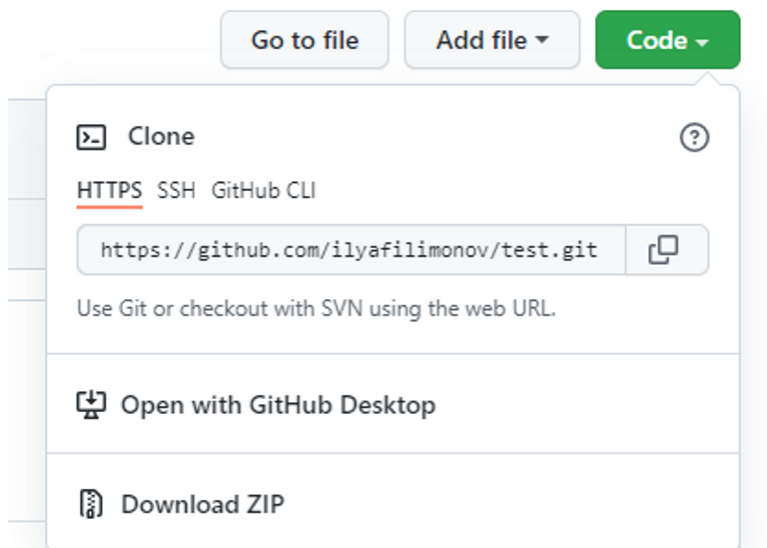
Заходим в нужный репозиторий, нажимаем на «вилку» с надписью fork. Форк репозитория создан и находится в вашем профиле на GitHub.

Теперь нужно клонировать форк себе на компьютер, чтобы вести работу с кодом локально. Тут нам и пригодится SSH.

Работа с изменениями

git clone

Необходима для клонирования репозитория. Перейдем на репозиторий на github.



Склонировать репозиторий также можно с помощью команды `git clone`.

```
user@DESKTOP-U05Q9LH MINGW64 ~/Downloads/itstep
$ git clone https://github.com/ilyafilimonov/test.git
Cloning into 'test'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (6/6), done.
```

Если вы правильно настроили SSH-ключи, Git начнёт процесс копирования репозитория на ваш компьютер. Если вы видите ошибку, в которой написано `Error: Permission denied (publickey)`, скорее всего, вы ошиблись где-то при выполнении инструкции по настройке SSH-ключа.

Если вы не хотите вручную вводить адрес репозитория, вы можете зайти на страницу проекта, нажать зелёную кнопку Clone or download (клонировать или скачать), выбрать Clone with SSH (клонировать по SSH) и скопировать адрес, который находится в текстовом поле. Этот адрес вы можете вставить в команду `git clone`.

Что такое markdown?

git add

Чтобы добавить отслеживание новых файлов, необходимо использовать команду `git add <filename> <filename>` для добавления нескольких файлов по имени.

```
i.filimonov@MED-FILIMONOV MINGW64 ~/Downloads/app (master)
$ git add -A

i.filimonov@MED-FILIMONOV MINGW64 ~/Downloads/app (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    deleted:   README.md
    new file:   app
    modified:  css/style.css
    deleted:   img/bg.jpg
    deleted:   img/logo.svg
    modified:  index.html
    new file:  js/script.js
```

Команда `git add -A` добавляет ещё и удалённые файлы, не только из текущей директории, но и из всего локального репозитория.

git status

После внесения любых изменений в проект можно посмотреть статус файлов с помощью команды `git status`. Она покажет файлы, в которых были произведены изменения, удалённые и новые, требующие добавления.

```
i.filimonov@MED-FILIMONOV MINGW64 ~/Downloads/app (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:   README.md
    modified:  css/style.css
    deleted:   img/bg.jpg
    deleted:   img/logo.svg
    modified:  index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    app/
    js/

no changes added to commit (use "git add" and/or "git commit -a")
```

git commit

После того как добавлены все новые файлы, можно делать фиксацию изменений. Фиксация изменений или коммит, очень важна, так как до выполнения этой команды ваши локальные изменения никуда не запишутся. Чтобы добавить коммит, необходимо ввести команду

```
git commit -m "Комментарий к коммиту"
```

```
i.filimonov@MED-FILIMONOV MINGW64 ~/Downloads/app (master)
$ git commit -m "Добавил базовую структуру файлов"
[master 81fda98] Добавил базовую структуру файлов
7 files changed, 29 insertions(+), 305 deletions(-)
delete mode 100644 README.md
create mode 160000 app
rewrite css/style.css (100%)
delete mode 100644 img/bg.jpg
delete mode 100644 img/logo.svg
rewrite index.html (91%)
create mode 100644 js/script.js
```

Стоит отметить, что необходимо правильно разбивать изменения на коммиты и давать полные комментарии к коммитам. Подробнее на эту тему можно почитать [здесь](#).

Если вы внесли изменения и хотите быстро их отменить, то воспользуйтесь командой `git reset`, которая отменяет все незафиксированные изменения.

По умолчанию, эта команда удаляет только из индекса. А команда `git reset --hard` безвозвратно удаляет незафиксированные текущие изменения из локального репозитория и из индекса.

git remote add

Поскольку все вышеописанные действия производятся в локальной копии репозитория, необходимо произвести связку локального репозитория с удаленным.

```
git remote add application https://github.com/ilyafilimonov/application.git
```

```
MINGW64:/c/Users/user/Downloads/application
user@DESKTOP-U05Q9LH MINGW64 ~/Downloads/application (master)
$ git remote add application https://github.com/ilyafilimonov/application.git
```

git push

Теперь внесенные нами изменения мы можем отправить на сервер, чтобы другие участники процесса смогли получить актуальную версию. Для этого есть команда

`git push origin main`, которая отправляет все зафиксированные изменения на удаленный репозиторий.

`origin` - название репозитория

`main` - название ветки

```
99915@Ilia MINGW64 ~/OneDrive/Desktop/otus (main)
$ git push otus main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 493 bytes | 493.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/iliafilimonov1/otus.git
   d651506..12b3c38  main -> main
```

После выполнения всех действий, обновим страницу на github и увидим, что наши локальные изменения были успешно перенесены на удаленный репозиторий.

git pull

Любая работа с изменениями начинается с получения последней версии удалённого репозитория. Получить последнюю версию можно с помощью команды `git pull`.

```
i.filimonov@MED-FILIMONOV MINGW64 ~/Downloads/app (master)
$ git pull
Already up to date.
```

Полезная ссылка: [git client fork](#)

Работа с ветками

Работая с Git, приходится постоянно создавать и перемещаться по веткам.

Команда `git checkout -b branch-name` создаст ветку с указанным именем и автоматически переключится на неё.

```
user@DESKTOP-U05Q9LH MINGW64 ~/Downloads/application (master)
$ git checkout -b task-1
Switched to a new branch 'task-1'
```

После создания ветку можно отправить на сервер с помощью команды `git push origin branch-name`.

```
user@DESKTOP-U05Q9LH MINGW64 ~/Downloads/application (task-1)
$ git push origin task-1
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'task-1' on GitHub by visiting:
remote:   https://github.com/ilyafilimonov/application/pull/new/task-1
remote:
To https://github.com/ilyafilimonov/application.git
 * [new branch]      task-1 -> task-1
```

Аналогично можно забрать себе на компьютер ветку с удалённого репозитория командой `git checkout origin/branch-name -b branch-name`.

Чтобы не хранить названия веток в памяти или не искать названия веток, существуют две специальные команды, которые позволяют посмотреть все существующие ветки локального репозитория `git branch` или все существующие ветки удалённого репозитория `git branch -r`.

```
user@DESKTOP-U05Q9LH MINGW64 ~/Downloads/application (task-1)
$ git branch -r
  application/master
  origin/master
  origin/task-1
```

Переключиться на любую локальную ветку можно с помощью команды `git checkout branch-name`.

```
user@DESKTOP-U05Q9LH MINGW64 ~/Downloads/application (task-1)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'application/master'.
```

`.gitignore`

Git рассматривает каждый файл в вашей рабочей копии как файл одного из трех нижеуказанных типов.

- отслеживаемый файл - файл, который был предварительно проиндексирован или зафиксирован в коммите.
- неотслеживаемый файл - файл, который *не был* проиндексирован или зафиксирован в коммите.
- игнорируемый файл - файл, явным образом помеченный для Git как файл, который необходимо игнорировать.

Игнорируемые файлы - это, как правило, артефакты сборки и файлы, генерируемые машиной из исходных файлов в вашем репозитории, либо файлы, которые по какой-либо иной причине не должны попадать в коммиты. Вот некоторые распространенные примеры таких файлов:

- кэши зависимостей, например содержимое `/node_modules` или `/packages`
- скомпилированный код, например файлы `.o` , `.class`
- каталоги для выходных данных сборки, например `/bin` , `/out` или `/target`
- файлы, сгенерированные во время выполнения, например `.log` , `.lock` или `.tmp`
- скрытые системные файлы, например `.DS_Store` или `Thumbs.db`
- личные файлы конфигурации IDE, например `.idea/workspace.xml`

Игнорируемые файлы отслеживаются в специальном файле `.gitignore` , который регистрируется в корневом каталоге репозитория. В Git нет специальной команды для указания игнорируемых файлов: вместо этого необходимо вручную отредактировать файл `.gitignore` , чтобы указать в нем новые файлы, которые должны быть проигнорированы.

Файлы `.gitignore` содержат шаблоны, которые сопоставляются с именами файлов в репозитории для определения необходимости игнорировать эти файлы.

Шаблоны игнорирования в Git

Для сопоставления с именами файлов в `.gitignore` используются шаблоны подстановки. С помощью различных символов можно создавать собственные шаблоны.

Шаблон	Примеры соответствия	Пояснение
<code>**/logs</code>	<code>logs/debug.log</code> <code>logs/monday/foo.bar</code> <code>build/logs/debug.log</code>	Добавьте в начало шаблона две звездочки, чтобы сопоставлять каталоги в любом месте репозитория.
<code>**/logs/debug.log</code>	<code>logs/debug.log</code> <code>build/logs/debug.log</code> <i>НО НЕ</i> <code>logs/build/debug.log</code>	Две звездочки можно также использовать для сопоставления файлов на основе их имени и имени родительского каталога.
<code>*.log</code>	<code>debug.log</code> <code>foo.log</code> <code>logs/debug.log</code>	Одна звездочка - это подстановочный знак, который может соответствовать как нескольким символам, так и ни одному.
<code>*.log!important.log</code>	<code>debug.log</code> <code>trace.log</code> <i>НО НЕ</i> <code>important.log</code> <code>logs/important.log</code>	Добавление восклицательного знака в начало шаблона отменяет действие шаблона. Если файл соответствует некоему шаблону, но при этом <i>также</i> соответствует отменяющему шаблону, указанному после, такой файл не будет игнорироваться.
<code>*.log!important/*.log</code> <code>trace.*</code>	<code>debug.log</code> <code>important/trace.log</code> <i>НО НЕ</i> <code>important/debug.log</code>	Шаблоны, указанные после отменяющего шаблона, снова будут помечать файлы как игнорируемые, даже если ранее игнорирование этих файлов было отменено.
<code>/debug.log</code>	<code>debug.log</code> <i>НО НЕ</i> <code>logs/debug.log</code>	Косая черта перед именем файла соответствует файлу в корневом каталоге репозитория.
<code>debug.log</code>	<code>debug.log</code> <code>logs/debug.log</code>	По умолчанию шаблоны соответствуют файлам, находящимся в любом каталоге
<code>debug?.log</code>	<code>debug0.log</code> <code>debugg.log</code> <i>НО НЕ</i> <code>debug10.log</code>	Знак вопроса соответствует строго одному символу.
<code>debug[0-9].log</code>	<code>debug0.log</code> <code>debug1.log</code> <i>НО НЕ</i> <code>debug10.log</code>	Квадратные скобки можно также использовать для указания соответствия одному символу из заданного диапазона.
<code>debug[01].log</code>	<code>debug0.log</code> <code>debug1.log</code> <i>НО НЕ</i> <code>debug2.log</code> <code>debug01.log</code>	Квадратные скобки соответствуют одному символу из указанного набора.
<code>debug[!01].log</code>	<code>debug2.log</code> <i>НО НЕ</i> <code>debug0.log</code> <code>debug1.log</code> <code>debug01.log</code>	Восклицательный знак можно использовать для указания соответствия любому символу, кроме символов из указанного набора.

<code>debug[a-z].log</code>	<code>debuga.log</code> <code>debugb.log</code> <i>но не</i> <code>debug1.log</code>	Диапазоны могут быть цифровыми или буквенными.
<code>logs</code>	<code>logslogs/debug.log</code> <code>logs/latest/foo</code> <code>.bar</code> <code>build/logsbuild/logs/debug.log</code>	Без косой черты в конце этот шаблон будет соответствовать и файлам, и содержимому каталогов с таким именем. В примере соответствия слева игнорируются и каталоги, и файлы с именем <code>logs</code>
<code>logs/</code>	<code>logs/debug.log</code> <code>logs/latest/foo.bar</code> <code>build/logs/foo.bar</code> <code>build/logs/latest/debug.log</code>	Косая черта в конце шаблона означает каталог. Все содержимое любого каталога репозитория, соответствующего этому имени (включая все его файлы и подкаталоги), будет игнорироваться
<code>logs/!logs/important.log</code>	<code>logs/debug.log</code> <code>logs/important.log</code>	Файл <code>logs/important.log</code> из примера слева не должен быть исключен из списка игнорируемых? Нет! Из-за странностей Git, связанных с производительностью, вы не можете отменить игнорирование файла, которое задано шаблоном соответствия каталогу
<code>logs/**/debug.log</code>	<code>logs/debug.log</code> <code>logs/monday/debug.log</code> <code>logs/monday/pm/debug.log</code>	Две звездочки соответствуют множеству каталогов или ни одному.
<code>logs/*day/debug.log</code>	<code>logs/monday/debug.log</code> <code>logs/tuesday</code> <code>/debug.log</code> <i>но не</i> <code>logs/latest/debug.log</code>	Подстановочные символы можно использовать и в именах каталогов.
<code>logs/debug.log</code>	<code>logs/debug.log</code> <i>но не</i> <code>debug.log</code> <code>build/logs/debug.log</code>	Шаблоны, указывающие на файл в определенном каталоге, задаются относительно корневого каталога репозитория.

Две звездочки (**) означают, что ваш файл `.gitignore` находится в каталоге верхнего уровня вашего репозитория, как указано в соглашении.

Общие файлы .gitignore в вашем репозитории

Обычно правила игнорирования Git задаются в файле `.gitignore` в корневом каталоге репозитория. Также вы можете определить несколько файлов `.gitignore` в разных каталогах репозитория. Каждый шаблон из конкретного файла `.gitignore` проверяется относительно каталога, в котором содержится этот файл. Однако проще всего (и этот подход рекомендуется в качестве общего соглашения) определить один файл `.gitignore` в корневом каталоге.

Персональные правила игнорирования в Git

В специальном файле, который находится в папке `.git/info/exclude`, можно определить персональные шаблоны игнорирования для конкретного репозитория. Этот файл не имеет контроля версий и не распространяется вместе с репозиторием, поэтому он хорошо подходит для указания шаблонов, которые будут полезны только вам. Например, если у вас есть пользовательские настройки для ведения журналов или специальные инструменты разработки, которые создают файлы в рабочем каталоге вашего репозитория, вы можете добавить их в `.git/info/exclude`, чтобы они случайно не попали в коммит в вашем репозитории.

Глобальные правила игнорирования в Git

Для всех репозиториев в локальной системе можно определить глобальные шаблоны игнорирования Git, настроив параметр конфигурации Git `core.excludesFile`. Этот файл нужно создать самостоятельно. Если вы не знаете, куда поместить глобальный файл `.gitignore`, расположите его в домашнем каталоге (потом его будет легче найти). После создания этого файла необходимо настроить его местоположение с помощью команды `git config`:

```
touch ~/.gitignore git config --global core.excludesFile ~/.gitignore
```

Будьте внимательны при указании глобальных шаблонов игнорирования, поскольку для разных проектов актуальны различные типы файлов. Типичные кандидаты на глобальное игнорирование - это специальные файлы операционной системы (например, `.DS_Store` и `thumbs.db`) или временные файлы, создаваемые некоторыми инструментами разработки.

Игнорирование ранее закоммиченного файла

Чтобы игнорировать файл, для которого ранее был сделан коммит, необходимо удалить этот файл из репозитория, а затем добавить для него правило в `.gitignore`. Используйте команду `git rm` с параметром `--cached`, чтобы удалить этот файл из репозитория, но оставить его в рабочем каталоге как игнорируемый файл.

```
echo debug.log >> .gitignore    git rm --cached debug.log rm 'debug.log'
git commit -m "Start ignoring debug.log"
```

Опустите опцию `--cached`, чтобы удалить файл как из репозитория, так и из локальной файловой системы.

Коммит игнорируемого файла

Можно принудительно сделать коммит игнорируемого файла в репозиторий с помощью команды `git add` с параметром `-f` (или `--force`):

```
cat .gitignore *.log    git add -f debug.log    git commit -
m "Force adding debug.log"
```

Этот способ хорош, если у вас задан общий шаблон (например, `*.log`), но вы хотите сделать коммит определенного файла. Однако еще лучше в этом случае задать исключение из общего правила:

```
echo !debug.log >> .gitignore    cat .gitignore *.log !debug.log
git add debug.log    git commit -m "Adding debug.log"
```

Этот подход более прозрачен и понятен, если вы работаете в команде.

Скрытие изменений в игнорируемом файле

Команда `git stash` - это мощная функция системы Git, позволяющая временно отложить и отменить локальные изменения, а позже применить их повторно. По умолчанию команда `git stash` ожидаемо не обрабатывает игнорируемые файлы и создает отложенные изменения только для тех файлов, которые отслеживаются Git. Тем не менее вы можете вызвать команду `git stash` с параметром `--all`, чтобы создать отложенные изменения также для игнорируемых и неотслеживаемых файлов.

Отладка файлов .gitignore

Если шаблоны `.gitignore` сложны или разбиты на множество файлов `.gitignore`, бывает непросто отследить, почему игнорируется определенный файл. Используйте команду `git check-ignore` с параметром `-v` (или `--verbose`), чтобы определить, какой шаблон приводит к игнорированию конкретного файла:

```
git check-ignore -v debug.log .gitignore:3:*.log debug.log
```

Вывод будет следующим:

```
<file containing the pattern> : <line number of the pattern> : <pattern> <file name>
```

При желании команде `git check-ignore` можно передать несколько имен файлов, причем сами имена могут даже не соответствовать файлам, существующим в вашем репозитории.

Шпаргалка по git.

Восстановление накопленных изменений

В том случае, если изменения, внесённые пользователем, находятся в режиме накопления, применить их к ветке можно с помощью команды `git stash apply`. Также можно запустить `git diff` — эта команда поможет выявить различия. Для того, чтобы затем избавиться от накопленных данных, нужно запустить команду:

```
git stash drop
```

Если существует более одного накопления, найти нужное можно с помощью команды:

`git stash list` затем можно применить его, воспользовавшись его индексом:

```
git stash@{1}
```

Необходимо учитывать, что отсчёт индексов ведётся от нуля.

Восстановление удалённого тега

В том случае, если необходимо восстановить случайно удалённый тег, начать можно с его поиска:

```
git fsck --unreachable | grep ta
```

После того, как нужный тег найден, его следует восстановить:

```
git update-ref refs/tags/название-тега
```

Восстановление удалённого файла

Если вы случайно удалили файл, его можно быстро восстановить:

```
git checkout myfile.txt
```

Если требуется восстановить файл из конкретной временной точки истории коммитов, следует узнать хеш нужного коммита и запустить команду:

```
git checkout $commit~1 myfile.txt
```

Восстановление удалённой ветки

С помощью команды `git reflog` можно узнать хеш (SHA1) последнего коммита в удалённой ветке. Скопируйте этот хеш и используйте в команде:

```
git checkout <sha>
```

После этого восстановить удалённую ветку можно будет вот такой командой:

```
git checkout -b <название-ветки>
```

Изменение сообщения коммита перед его отправкой

Изменить сообщение коммита можно с помощью команды `git commit --amend`, она откроет редактор, в котором можно будет внести необходимые поправки в последнее сообщение.

Сообщение можно изменить и напрямую с помощью команды

```
git commit --amend -m "Новое прекрасное сообщение"
```

Изменение сообщения коммита после его отправки

В данном случае процесс занимает два шага. Сначала нужно изменить сообщение с помощью команды `git commit --amend`, а затем перезаписать историю коммитов локальной ветки: `git push <remote> <branch> --force`

Предупреждение: подобная насильная перезапись может привести к потере коммитов из внешней ветки, если с ней давно не было синхронизации, соблюдайте осторожность.

Коммит в неправильную ветку

Нужно переключиться на новую ветку, которую вы забыли предварительно создать:

```
git checkout -b название-новой-ветки.
```

А затем переключиться к оригинальной ветке:

```
git checkout название-оригинальной-ветки
```

...и «откатиться» до последнего коммита, который нужно сохранить.

Чтобы это сделать, можно воспользоваться командой `git log` и сохранить хеш (SHA1) последнего коммита, который нужно оставить.. Например, это `a31a45c`.

Теперь его нужно сбросить: `git reset --hard a31a45c` и отправить получившийся результат.

Предупреждение: Убедитесь в том, что никто не отправлял коммиты в оригинальную ветку во время выполнения вышеописанных процедур, в противном случае эти изменения будут потеряны!

Откат к конкретному коммиту в истории

Если вас не очень беспокоят изменения в локальном репозитории, то можно «откатиться» к конкретному коммиту в истории с помощью команды:

```
git reset --hard HEAD~1
```

Эта команда установит HEAD на конкретный коммит. Также можно воспользоваться хешем коммита.

Отмена коммита до публикации изменений

Если вы сделали коммит, который впоследствии понадобилось отредактировать или полностью стереть, поможет команда `git reset`.

```
git reset HEAD~1 # отменить последний коммит, сохранить изменения
git reset --hard HEAD~1 # отменить последний коммит, стереть изменения
```

Будьте осторожны используя второй вариант, поскольку изменения ваших локальных файлов будут потеряны.

Чтобы сохранить сообщение коммита, наберите :

```
git commit -i ORIG_HEAD
```

Отмена коммита после отправки его в master-репозиторий

Рассмотрим процедуру возврата одного или нескольких коммитов, которые нужно стереть из удалённой ветки. Обозначить конкретный коммит можно с помощью его хеша:

```
git revert b712c3c
```

Отмена только коммита, который является вторым после последнего:

```
git revert HEAD^
```

Простая отмена последнего коммита:

```
git revert -n HEAD
```

Отмена локальных изменений файлов

Простейшим способом избавиться от нежелательных изменений для файлов и папок является восстановление состояния последнего коммита. Сделать это можно с помощью специальной команды:

```
git checkout myfile.txt
```

Кроме того, можно восстановить конкретный путь к файлу:

```
git checkout -- путь-до-файла
```

Отображение всех коммитов одного файла

Если вы хотите просмотреть все коммиты с изменениями конкретного файла, воспользуйтесь командой `git log --follow -p -- myfile`

Аргумент `—follow` позволяет вывести все изменения над файлом, даже если в процессе работы он был переименован.

Если опустить опцию `-p`, то система выведет только сообщения коммитов, но не их содержимое.

Переименование локальной и удалённой ветки

Предложим, у вас есть ветка «*fix-bug25*», которую вы захотели переименовать в «*hotfix-users*». Прежде всего, нужно будет изменить локальную ветку:

```
git branch -m fix-bug25 hotfix-users
```

А затем — удалённую ветку: переименовать её напрямую нельзя, поэтому нужно будет её удалить, и затем опубликовать заново уже с новым именем.

Прежде чем приступить к этим процедурам, следует убедиться, что никто из членов команды не работает с этой веткой! Удаляем ветку: `git push origin :fix-bug25`

А теперь заново публикуем её с новым именем: `git push origin hotfix-users`

Переименование тега

Чтобы переименовать существующий тег:

```
git tag новое-название-тега старое-название-тега git tag -d старое-название-тега git push origin :refs/tags/старое-название-тега git push --tags
```

Перестать отслеживать существующие файлы

Если вы хотите перестать отслеживать файлы, которые уже есть в репозитории, но при этом желаете сохранить его локально, осуществите коммит изменений и запустите команду:

```
git rm -r --cached
```

Она удалит изменённые файлы из зоны подготовленных файлов (staging area). Затем нужно запустить команду:

```
git add .
```

и отправить изменения.

Подготовка удалённых файлов

Чтобы подготовить к коммиту файлы и папки, которые были удалены локально, можно использовать специальную команду:

```
git add -u
```

Если требуется подготовить только используемый в данный момент путь, воспользуйтесь командой

```
git add -u .
```

Просмотр всех неотправленных коммитов

Чтобы просмотреть все коммиты, которые ещё не были отправлены в соответствующие ветки, воспользуйтесь следующей командой:

```
git log --branches --not --remotes
```

Кроме того, можно использовать:

```
git log origin/master..HEAD
```

Просмотр старой ревизии файла

Существует возможность просмотреть содержимое файла в конкретный момент времени в прошлом. Для этого нужно использовать команду:

```
git show commitHash:myfile.txt
```

Публикация локальной ветки для удалённого редактирования

Если вы создали локальную ветку, и хотите, чтобы другие пользователи могли с ней работать, воспользуйтесь командой:

```
git push -u origin название-моей-новой-ветки
```

Теперь они тоже смогут вносить изменения в эту ветку.

Сброс локальной ветки до состояния удалённой

В том случае, если отсутствуют изменения, которые необходимо сохранить, сбросить локальную ветку до состояния удалённой можно с помощью двух простых команд.

Прежде всего нужно получить свежие обновления из удалённой ветки:

```
git fetch название-удалённой-ветки.
```

А затем нужно сообщить git, что локальную ветку следует «откатить» до состояния удалённой:

```
git reset --hard origin/название-локальной-ветки.
```

При наличии коммита, который нужно сохранить, перед сбросом нужно создать новую ветку и произвести коммит: `git commit -m «Обновление»`

```
git branch название-новой-ветки
```

Синхронизировать ветку с master-репозиторием

Чтобы синхронизировать последние изменения в репозитории master (или с любой другой веткой, с которой вы работали) необходимо «перебазировать» локальную ветку. Предположим, вы работаете над веткой foobar:

```
git checkout foobar
```

А затем осуществляете «перебазирование»:

```
git rebase master
```

После этого будут применены коммиты origin из master. После разрешения конфликтов процесс можно продолжить с помощью команды `git rebase —continue`. Теперь можно продолжать работу над своей веткой или осуществить её слияние (merge) с главным репозиторием.

Слияние локальных изменений с другой веткой

Это можно сделать прямо в процессе стандартного слияния (merge). Вам стоит *сохранить историю слияний* используя флаг `—no-ff`, что означает *no fast forward*.

Перейдите в ветку, в которую будут вливаться изменения, убедитесь в её актуальности и запустите процесс:

```
git merge <другая-ветка> --no-ff
```

Затем появится сообщение о коммите `merge X into Y branch`, после чего вы можете смело запустить ваше слияние.>

Создание новой ветки с изменениями текущей

Часто возникает ситуация, при которой пользователи начинают изменять файлы в ветке, чтобы что-то исправить, и лишь позднее вспоминают, что предварительно не создали новую ветку. К счастью, есть способ сделать это уже в процессе: